Dictionaries and Their Implementations

Chapter 18

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Dictionary

- Consider need to search such a collection for
  - Name
  - Address
  - Population by City, E.g. population of London
    - Binary search on City (sorted)
  - Population of cities in Spain
    - Scan entire collection
- Criterion chosen for search is *search key*
- The ADT dictionary uses a search key to identify its entries

City	Country	Population
Buenos Aires	Argentina	13,639,000
Cairo	Egypt	17,816,000
Johannesburg	South Africa	7,618,000
London	England	8,586,000
Madrid	Spain	5,427,000
Mexico City	Mexico	19,463,000
Mumbai	India	16,910,000
New York City	U.S.A.	20,464,000
Paris	France	10,755,000
Sydney	Australia	3,785,000
Tokyo	Japan	37,126,000
Toronto	Canada	6,139,000

FIGURE 18-1 A collection of data about certain cities

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Dictionary

```
Dictionary
---------------------------------------------------------------
---------------------------------------------------------------
+isEmpty(): boolean
+getNumberOfEntries(): integer
+add(searchKey: KeyType, newValue: ValueType): boolean
+remove(targetKey: KeyType): boolean
+clear(): void
+getValue(targetKey: KeyType): ValueType
+contains(targetKey: KeyType): boolean
+traverse(visit(value: ValueType): void): void
```

FIGURE 18-2 UML diagram for a class of dictionaries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

- Categories of linear implementations
  - Sorted by search key array-based
  - Sorted by search key link-based

  insert a new item into proper position based on the value of search key

  - Unsorted array-based
  - Unsorted link-based

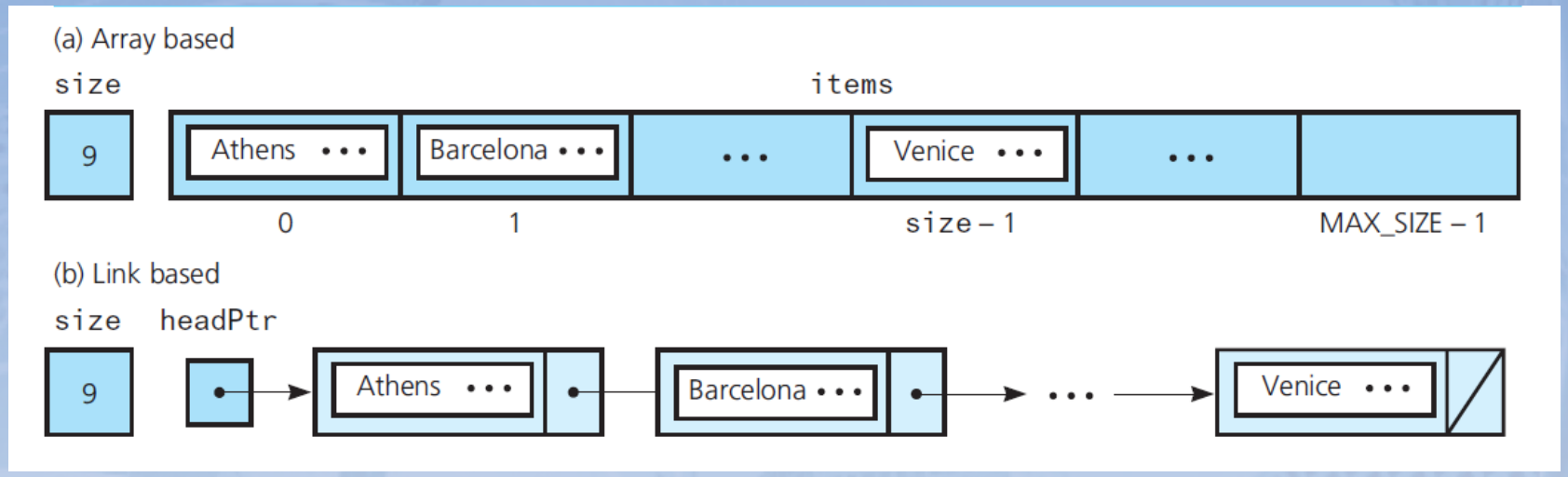
  insert a new item into any convenient location

Search key	Data Value

FIGURE 18-3 A dictionary entry

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations



- A linear array-based implementation of the ADT dictionary must shift data during an insertion or deletion in sorted order, which can be expensive, particularly for large dictionaries.
- A linear link-based implementation of the ADT dictionary eliminates the need to shift data, it does not support the insertion or removal operations any more efficiently than an array-based implementation, because you cannot perform a binary search in a reasonable fashion.

Figure 18-4 Data members for two sorted linear implementations of the ADT dictionary for the data in Figure 18-1

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

Another option is implementing it as a binary search tree.

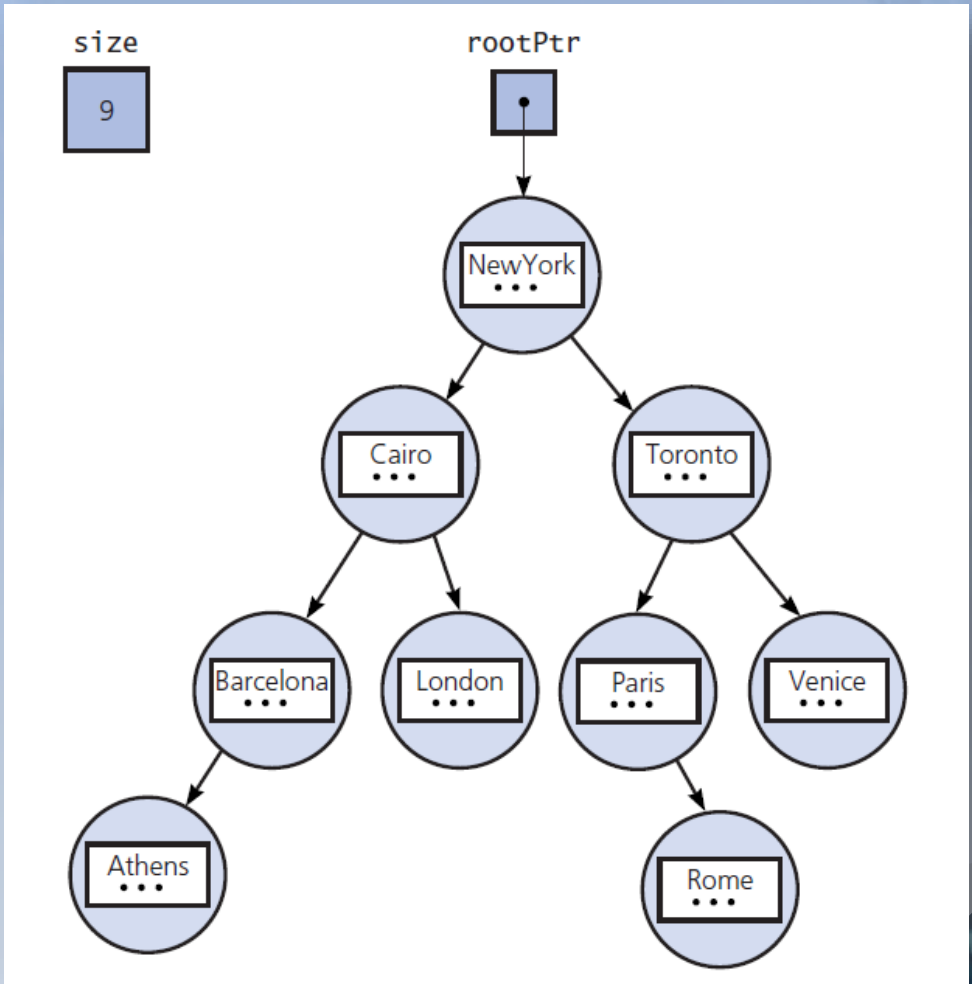


FIGURE 18-5 The data members for a binary search tree implementation of the ADT dictionary for the data in Figure 18-1

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Selecting an Implementation

- Linear implementations
  - Perspective
  - Efficiency
  - Motivation
- Consider
  - What operations are needed
  - How often each operation is required

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios

A. Addition and traversal in no particular order
   - Unsorted order is efficient
   - Array-based versus pointer-based

B. Retrieval
   - Sorted array-based can use binary search
   - Binary search impractical for link-based
   - Max size of dictionary affects choice

C. Addition, removal, retrieval, traversal in sorted order
   - Add and remove need to find position, then add or remove from that position
   - Array-based best for find, link-based best for addition/removal

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios

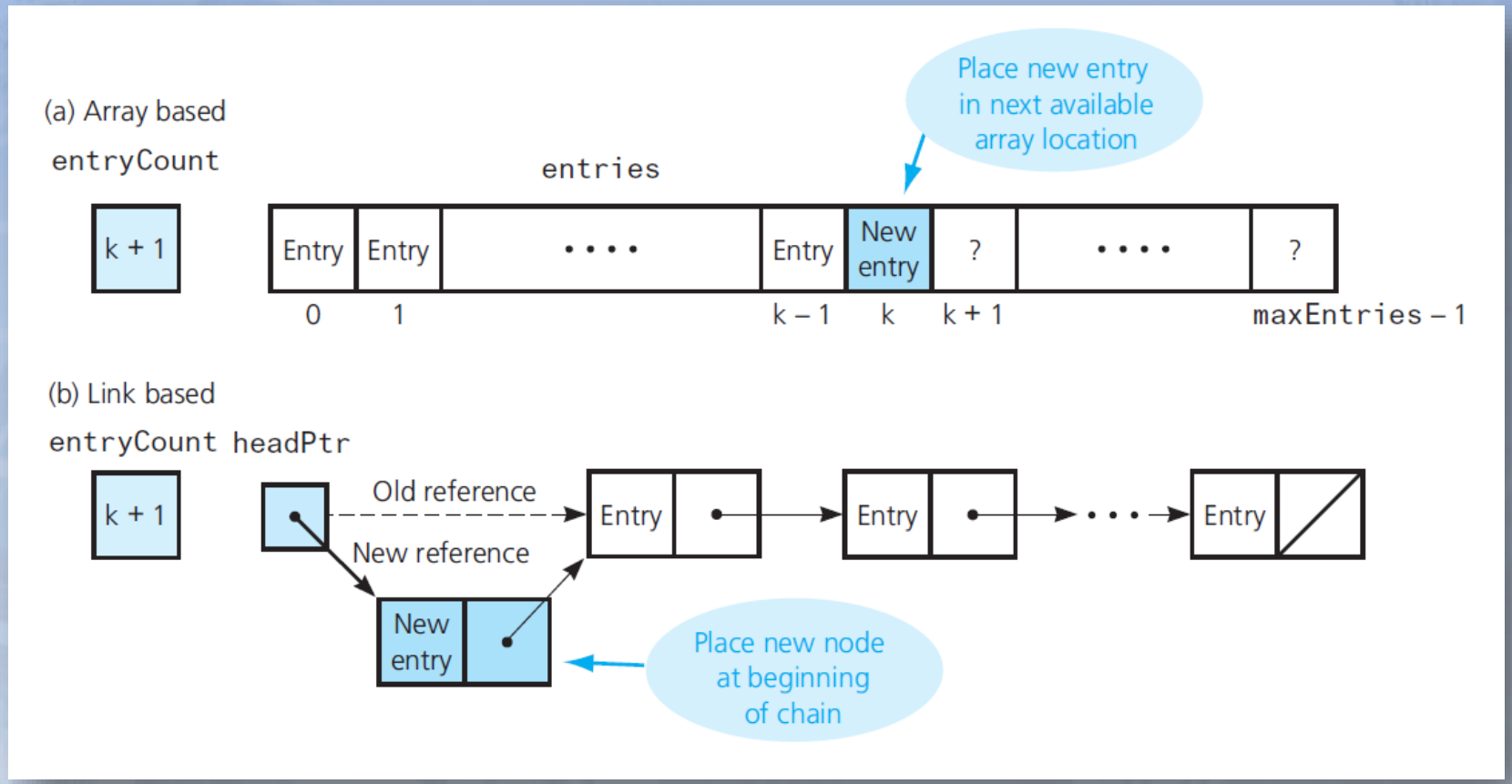


FIGURE 18-6 Addition for unsorted linear implementations

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios

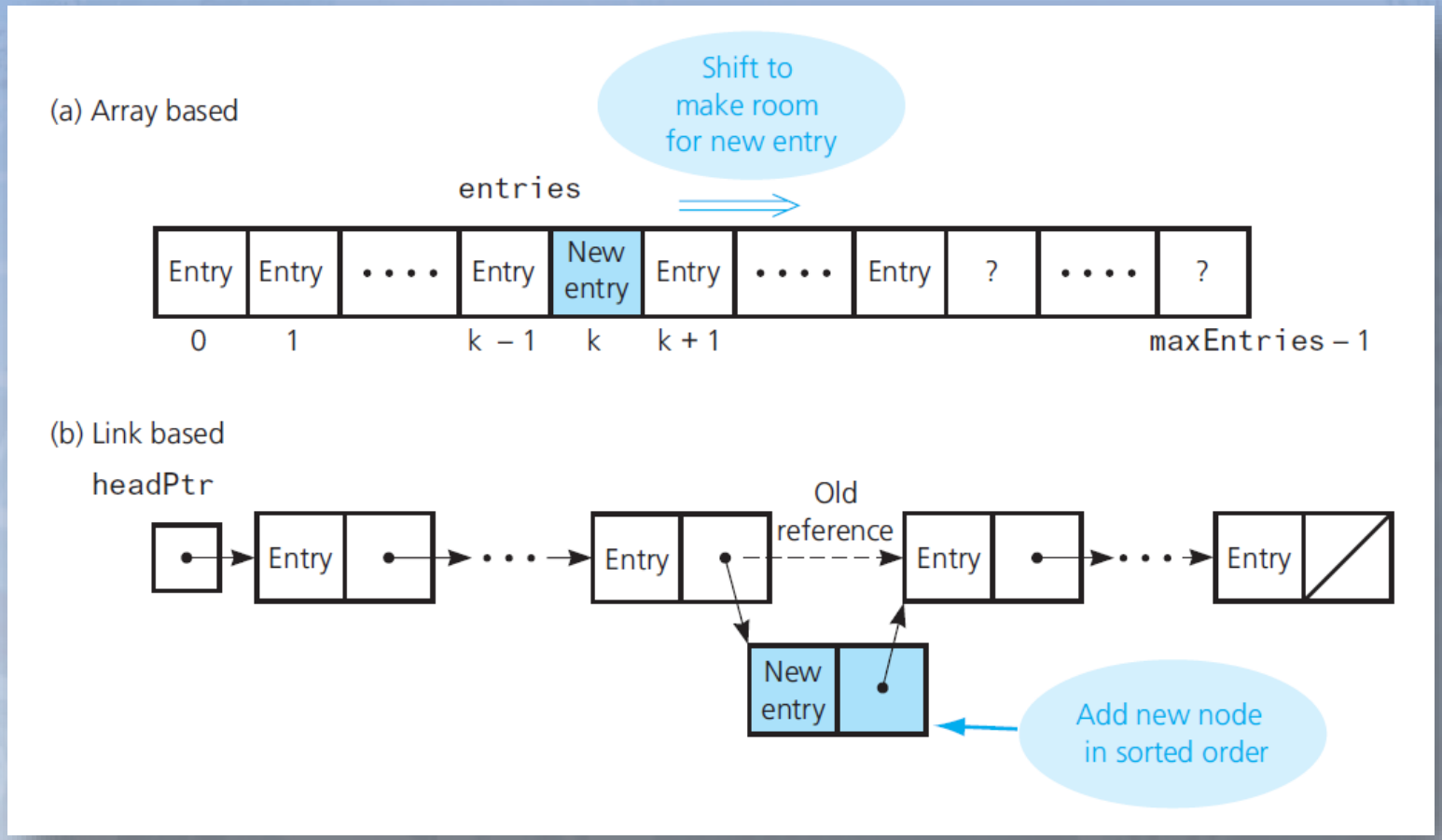


FIGURE 18-7 Addition for sorted linear implementations

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios

	Addition	Removal	Retrieval	Traversal
Unsorted array-based	O(1)	O(*n*)	O(*n*)	O(*n*)
Unsorted link-based	O(1)	O(*n*)	O(*n*)	O(*n*)
Sorted array-based	O(*n*)	O(*n*)	O(log *n*)	O(*n*)
Sorted link-based	O(*n*)	O(*n*)	O(*n*)	O(*n*)
Binary search tree	O(log *n*)	O(log *n*)	O(log *n*)	O(*n*)

FIGURE 18-8 The average-case order of the ADT dictionary operations for various implementations

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

- Situations occur for which search-tree implementations are not adequate.
- Consider a method which acts as an “address calculator” which determines an array index
  - Used for add, getValue, remove operations
- Called a hash function
  - Tells where to place item in a hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

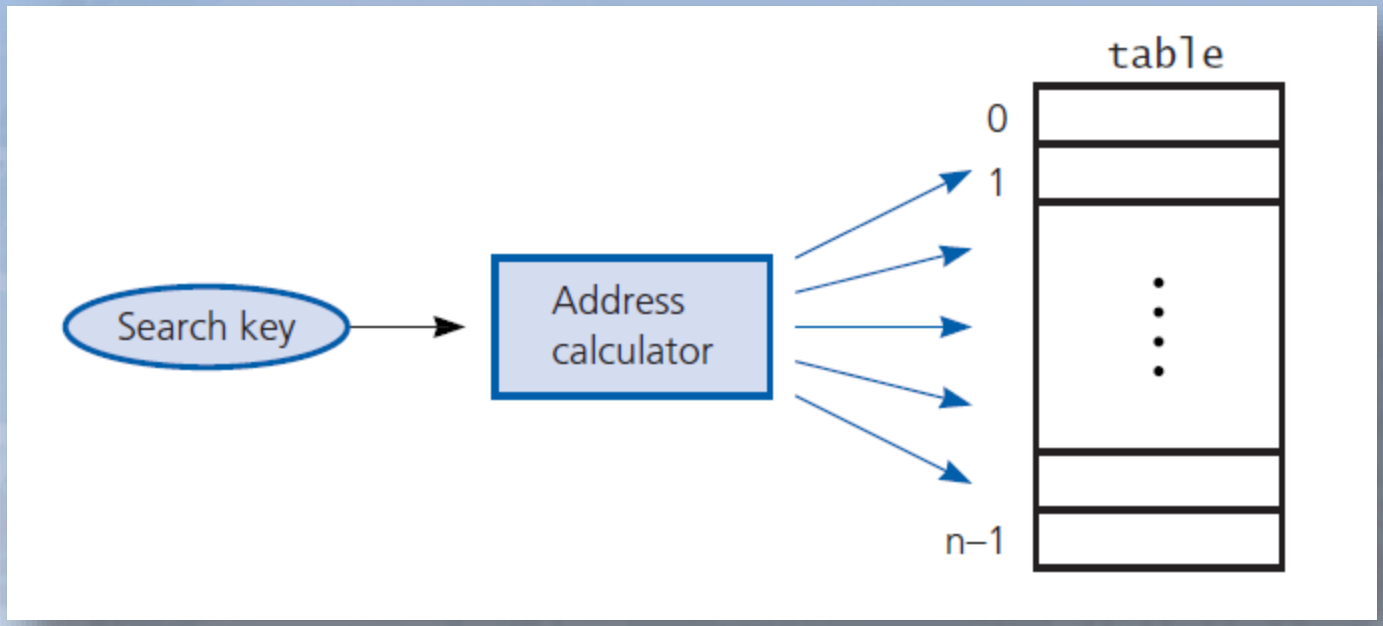


FIGURE 18-9 Address calculator

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

- Perfect hash function
  - Maps each search key into a unique location of the hash table
  - Possible if you know all the search keys
- Collision occurs when hash function maps more than one entry into same array location
- Hash function should
  - Be easy, fast to compute
  - Place entries evenly throughout hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hash Functions

- Sufficient for hash functions to operate on integers – examples:
  - Select digits from an ID number
    - Generally uneven distribution
  - Folding – add digits, sum is the table location
  - Modulo arithmetic $h(x) = x \bmod tableSize$
    - Simple, effective
    - Use prime number for table size
  - Convert character string to an integer – use ASCII values
    - Address potential overflow

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing

- Approach 1: Open addressing
  - Linear probing
  - Quadratic probing
  - Double hashing
  - Increase size of hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing

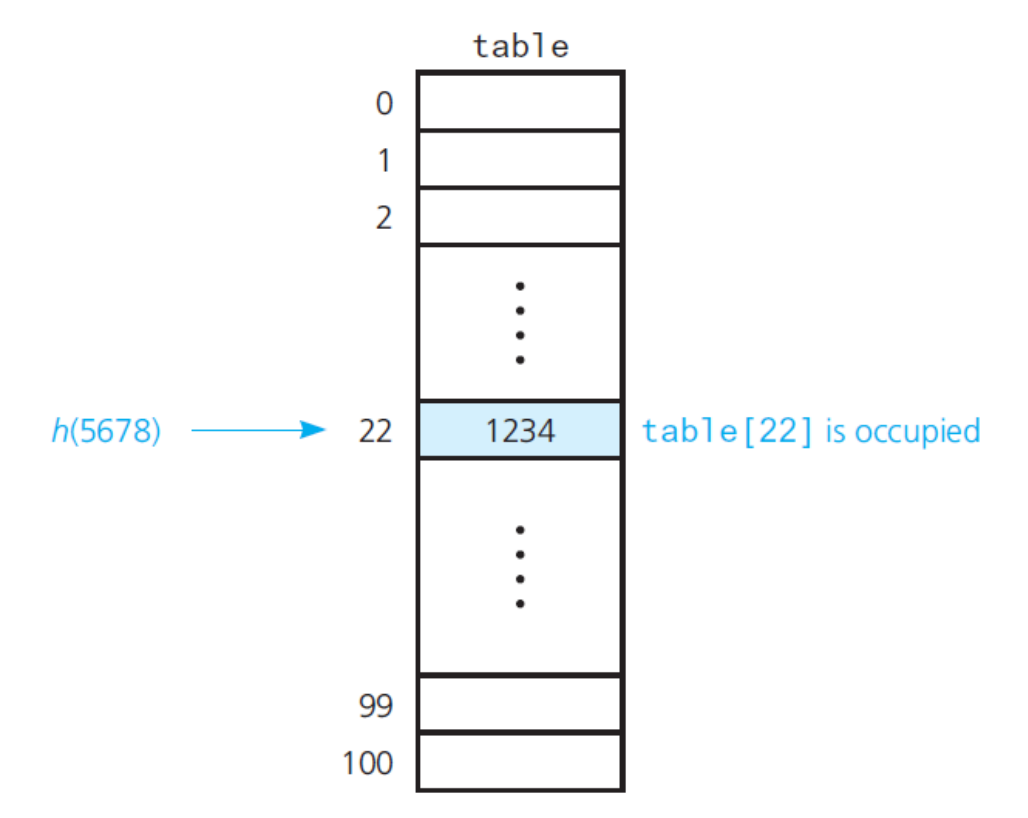
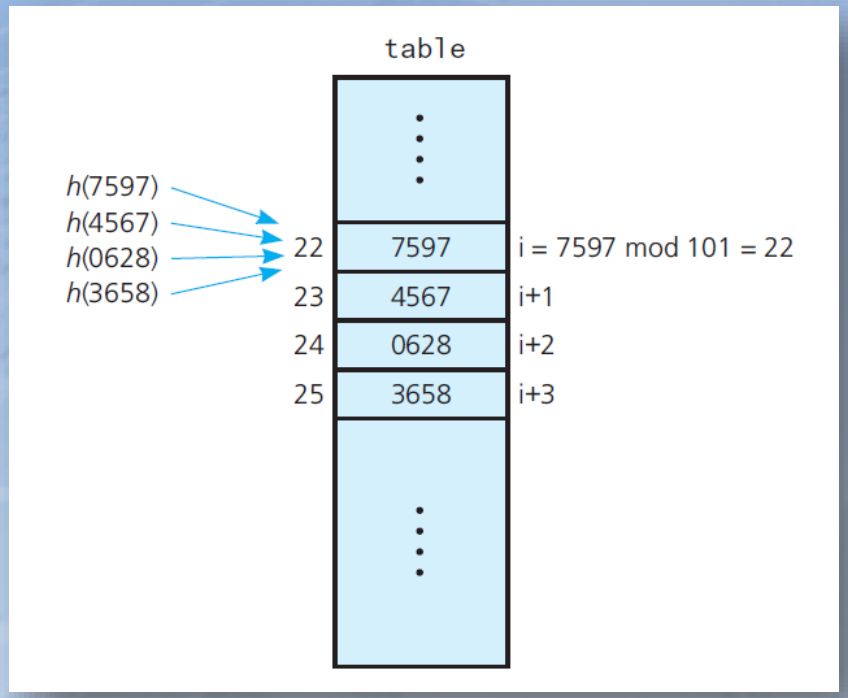


FIGURE 18-10 A collision

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



Linear probing

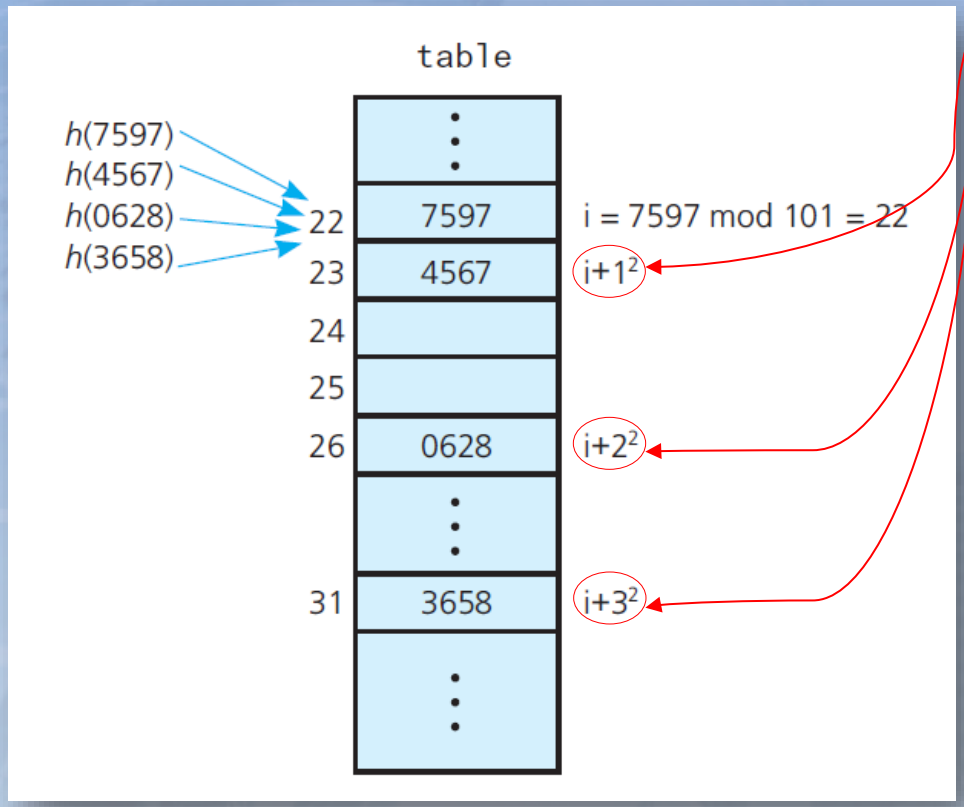
- Add - search the hash table, starting from the original hash location; if table[h(searchKey)] is occupied, check table[h(searchKey)+1] , table[h(searchKey)+2], … until you find an available location.
- Delete –place table location into one of three states:
  - occupied (currently in use),
  - empty (has not been used), or
    - Modify addItem to insert at this location
  - removed (was once occupied but is now available)
    - Modify getItem to continue probing
    - Modify addItem to insert at this location

- Items tend to cluster together in the hash table, when using this approach - primary clustering.
- Clusters can get close to one another and, merge into a larger cluster.
- Large clusters tend to get even larger

FIGURE 18-11 Linear probing with h(x) = x mod 101

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



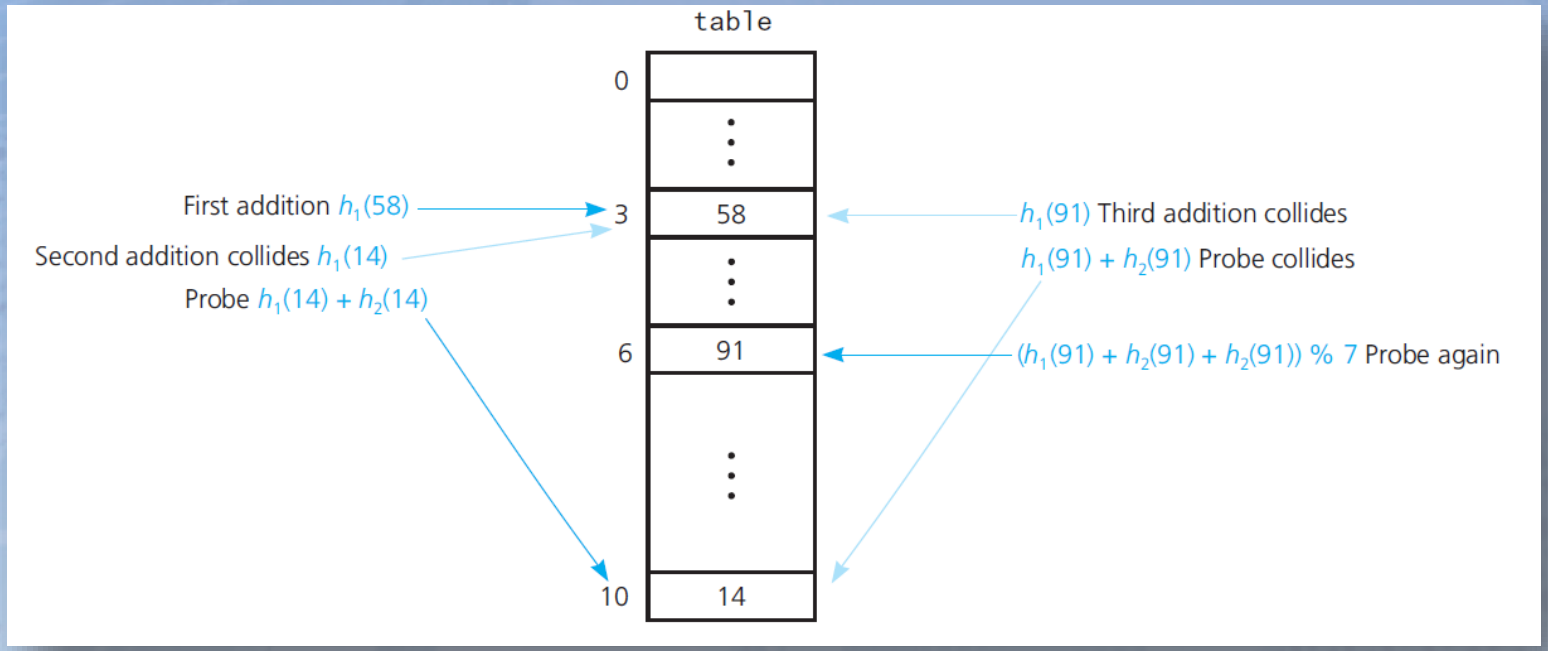
Quadratic probing.

- You can virtually eliminate primary clusters by adjusting the linear probing scheme.
- When two items hash collide, quadratic probing uses the same probe sequence for each item.
- The resulting phenomenon—called secondary clustering —delays the resolution of the collision.
- Although the research of quadratic probing remains incomplete, it appears that secondary clustering is not a problem.

FIGURE 18-12 Quadratic probing with h(x) = x mod 101

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



Double hashing - defines key-dependent probe sequences.

- searches the hash table in a linear order, starting at the location h1(key),
- second hash function h2 determines the size of the steps taken
  - h2(key) <> 0
  - h2 <> h1
- For example, let h1 and h2 be the primary and secondary hash functions defined as
  - h1 (key) = key mod 11 // 11 is table size and prime number
  - h2(key) = 7 (key mod 7) // visits all the table locations (table size and step are prime)

FIGURE 18-13 Double hashing during the addition of 58, 14, and 91

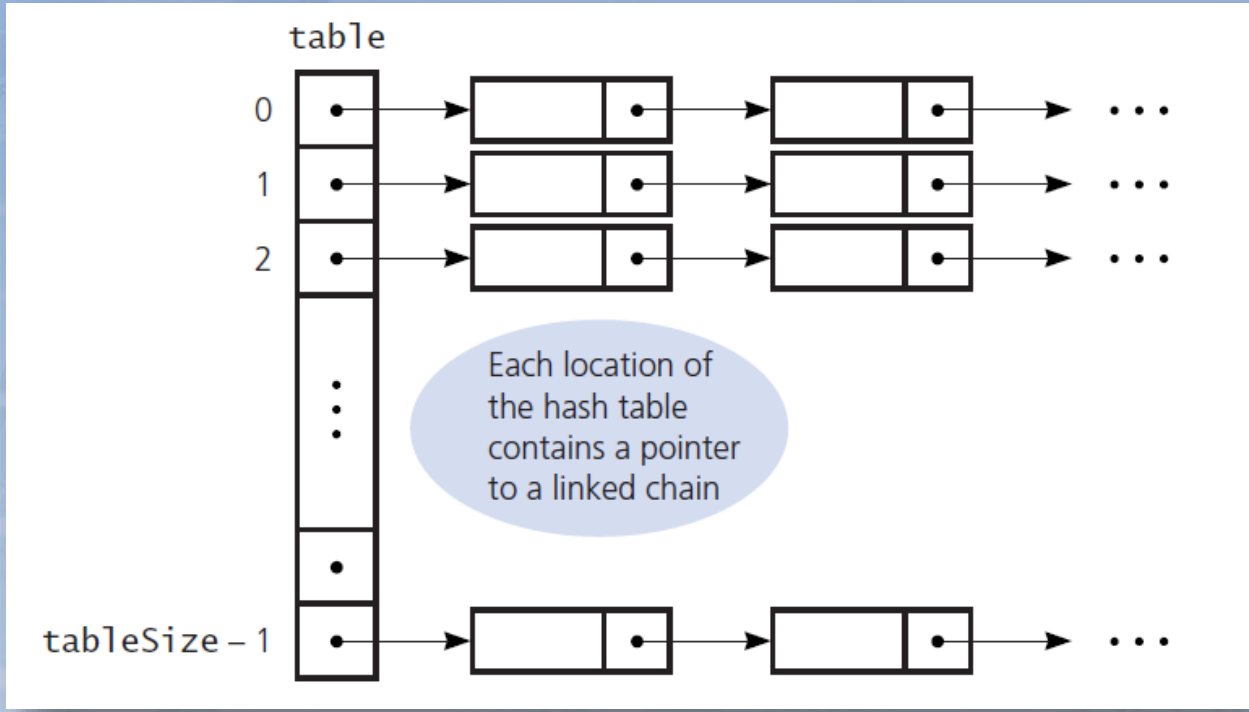
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions - Table Restructure

- You should not double the size of the array, since the size of the hash table should be prime.
- You do not move the items from the original hash table to the new hash table, since your hash function may have been impacted, e.g., x mod tableSize
- You need to apply your new hash function to every item in the old hash table before placing it into the new hash table. (Rehashing); expensive
- Approach 2: Resolving collisions by restructuring the hash table
  - Buckets
    - Define each has table location table[i] as an array, aka, bucket, where you store the collided items into in this array
    - Too small – only postponed collision problem
    - Too big – wasted storage
  - Separate chaining

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions - Table Restructure



Separate chaining. Design the hash table as an array of linked chains

- Each entry table[i] is a pointer to a chain of linked nodes containing the items that the hash function has mapped into location i
- The size of the dictionary is dynamic and can exceed the size of the hash table, because each linked chain can be as long as necessary.
- The most time-efficient collision-resolution scheme.

FIGURE 18-14 Separate chaining

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing

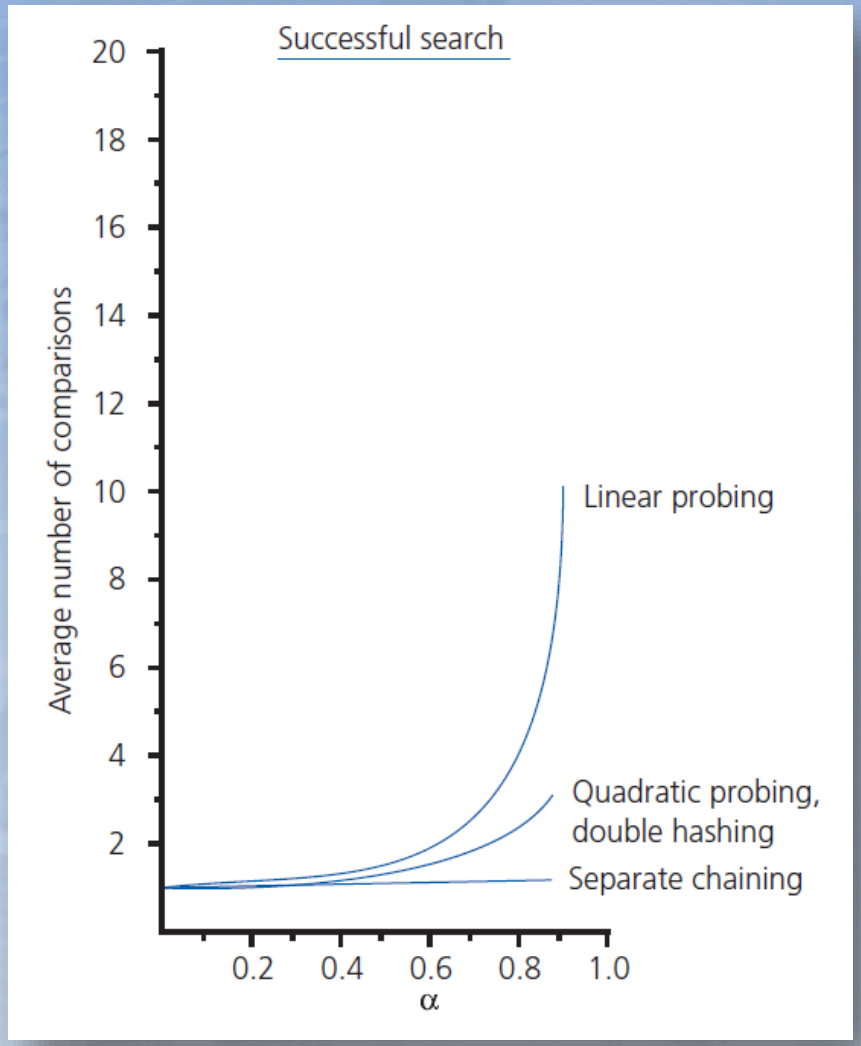
- Load factor measures how full a hash table is

$$\alpha = \frac{\text{Current number of table entries}}{\text{tableSize}}$$

- Unsuccessful searches
  - Generally require more time than successful
- Do not let the hash table get too full

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing



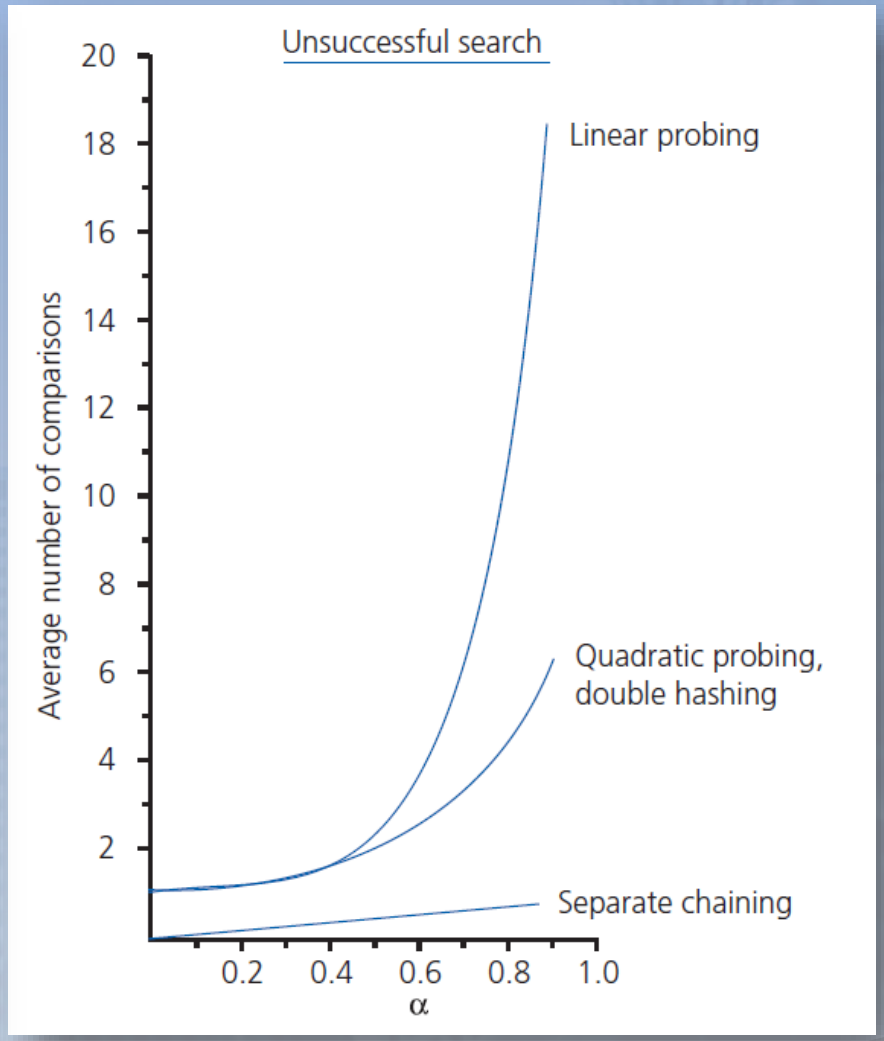


FIGURE 18-15 The relative efficiency of four collision-resolution methods

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

What Constitutes a Good Hash Function?

- Is hash function easy and fast to compute?
- Does hash function scatter data evenly throughout hash table?
- How well does hash function scatter random data?
- How well does hash function scatter *non*-random data?

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Dictionary Traversal: An Inefficient Operation Under Hashing

- Entries hashed into table[i] and table[i+1] have no ordering relationship
- Hashing does not support well traversing a dictionary in sorted order
  - Generally better to use a search tree
- In external storage possible to see
  - Hashing implementation of getValue
  - And search-tree for ordered operations simultaneously

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Chapter 18

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Chapter 18

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Dictionary

- Consider need to search such a collection for
  - Name
  - Address
- Criterion chosen for search is *search key*
- The ADT dictionary uses a search key to identify its entries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

```
/** An interface for the ADT dictionary.
 @file DictionaryInterface.h */

#ifndef DICTIONARY_INTERFACE_
#define DICTIONARY_INTERFACE_

#include "NotFoundException.h"

template<class KeyType, class ValueType>
class DictionaryInterface
{
public:
   /** Sees whether this dictionary is empty.
    @return  True if the dictionary is empty;
       otherwise returns false. */
   virtual bool isEmpty() const = 0;

   /** Gets the number of entries in this dictionary.
    @return  The number of entries in the dictionary. */
   virtual int getNumberOfEntries() const = 0;
```

LISTING 18-2 A header file for a class of dictionary entries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

```
    /** Adds a new search key and associated value to this dictionary.
     @pre  The new search key differs from all search keys presently
         in the dictionary.
     @post  If the addition is successful, the new key-value pair is in its
         proper position within the dictionary.
     @param searchKey  The search key associated with the value to be added.
     @param newValue  The value to be added.
     @return  True if the entry was successfully added, or false if not. */
    virtual bool add(const KeyType& searchKey, const ValueType& newValue) = 0;

    /** Removes a key-value pair from this dictionary.
     @post  If the entry whose search key equals searchKey existed in the
       dictionary, the entry was removed.
     @param searchKey  The search key of the entry to be removed.
     @return  True if the entry was successfully removed, or false if not. */
    virtual bool remove(const KeyType& searchKey) = 0;

    /** Removes all entries from this dictionary. */
    virtual void clear() = 0;
```

LISTING 18-2 A header file for a class of dictionary entries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

```
    virtual void clear() = 0;

    /** Retrieves the value in this dictionary whose search key is given
     @post  If the retrieval is successful, the value is returned.
     @param searchKey  The search key of the value to be retrieved.
     @return  The value associated with the search key.
     @throw  NotFoundException if the key-value pair does not exist. */
    virtual ValueType getValue(const KeyType& searchKey) const
                               throw (NotFoundException) = 0;

    /** Sees whether this dictionary contains an entry with a given search key.
     @post  The dictionary is unchanged.
     @param searchKey  The given search key.
     @return  True if an entry with the given search key exists in the
       dictionary. */
    virtual bool contains(const KeyType& searchKey) const = 0;
```

LISTING 18-2 A header file for a class of dictionary entries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

```
55     virtual bool contains(const KeyType& searchKey) const = 0;
56
57     /** Traverses this dictionary and calls a given client function once
58         for each entry.
59      @post  The given function's action occurs once for each entry in the
60         dictionary and possibly alters the entry.
61      @param visit  A client function. */
62     virtual void traverse(void visit(ValueType&)) const = 0;
63     /** Destroys this dictionary and frees its assigned memory. */
64
65     virtual ~DictionaryInterface(){ }
66   }; // end DictionaryInterface
67   #endif
```

LISTING 18-2 A header file for a class of dictionary entries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

```
/** A class of entries to add to an array-based implementation of the
 ADT dictionary.
 @file Entry.h */

#ifndef ENTRY_
#define ENTRY_

template <class KeyType, class ValueType>
class Entry
{
private:
   KeyType key;
   ValueType value;

protected:
   void setKey(const KeyType& searchKey);
```

Listing 18-2 A header file for a class of dictionary entries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

```
    void setKey(const KeyType& searchKey);

public:
    Entry();
    Entry(const KeyType& searchKey, const ValueType& newValue);
    ValueType getValue() const;
    KeyType getKey() const;
    void setValue(const ValueType& newValue);

    bool operator==(const Entry<KeyType, ValueType>& rightHandValue) const;
    bool operator>(const Entry<KeyType, ValueType>& rightHandValue) const;
}; // end Entry
#include "Entry.cpp"
#endif
```

Listing 18-2 A header file for a class of dictionary entries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorted Array-Based Implementation of ADT Dictionary

```
public:
   ArrayDictionary();
   ArrayDictionary(int maxNumberOfEntries);
   ArrayDictionary(const ArrayDictionary<KeyType, ValueType>& dictionary);

   virtual ~ArrayDictionary();

   bool isEmpty() const;
   int getNumberOfEntries() const;
   bool add(const KeyType& searchKey, const ValueType& newValue) throw(PrecondViolatedExcept);
   bool remove(const KeyType& searchKey);
   void clear();
   ValueType getValue(const KeyType& searchKey) const throw(NotFoundException);
   bool contains(const KeyType& searchKey) const;

   /** Traverses the entries in this dictionary in sorted search-key order
       and calls a given client function once for the value in each entry. */
   void traverse(void visit(ValueType&)) const;
}; // end ArrayDictionary
#include "ArrayDictionary.cpp"
#endif
```

LISTING 18-3 A header file for the class ArrayDictionary

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Binary Search Tree Implementation of the ADT Dictionary

```
/** A binary search tree implementation of the ADT dictionary
 that organizes its data in sorted search-key order.
 Search keys in the dictionary are unique.
 @file TreeDictionary.h */

#ifndef TREE_DICTIONARY_
#define TREE_DICTIONARY_

#include "DictionaryInterface.h"
#include "BinarySearchTree.h"
#include "Entry.h"
#include "NotFoundException.h"
#include "PrecondViolatedExcept.h"

template <class KeyType, class ValueType>
class TreeDictionary : public DictionaryInterface<KeyType, ValueType>
{
private:
```

LISTING 18-4 A header file for the class TreeDictionary

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Binary Search Tree Implementation of the ADT Dictionary

```
{
private:
   // Binary search tree of dictionary entries
   BinarySearchTree<Entry<KeyType, ValueType> > entryTree;

public:
   TreeDictionary();
   TreeDictionary(const TreeDictionary<KeyType, ValueType>& dictionary);

   virtual ~TreeDictionary();

   // The declarations of the public methods appear here and are the
   // same as given in Listing 18-3 for the class ArrayDictionary.
   ...
}; // end TreeDictionary
#include "TreeDictionary.cpp"
#endif
```

LISTING 18-4 A header file for the class TreeDictionary

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Binary Search Tree Implementation of the ADT Dictionary

```
template < class KeyType, class ValueType>
bool TreeDictionary<KeyType, ValueType>::add(const KeyType& searchKey,
                                             const ValueType& newValue)
                                             throw(PrecondViolatedExcept)
{
   Entry<KeyType, ValueType> newEntry(searchKey, newValue);

   // Enforce precondition: Ensure distinct search keys
   if (!itemTree.contains(newEntry))
   {
      // Add new entry and return boolean result
      return itemTree.add(Entry<KeyType, ValueType>(searchKey, newValue));
   }
   else
   {
      auto message = "Attempt to add an entry whose search key exists in dictionary.";
      throw(PrecondViolatedExcept(message)); // Exit the method
   }  // end if
} // end add
```

Method add which prevents duplicate keys.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing

- Linear probing – average number of comparisons

$$\frac{1}{2}\left[1 + \frac{1}{1-\alpha}\right] \quad \text{for a successful search, and}$$

$$\frac{1}{2}\left[1 + \frac{1}{(1-\alpha)^2}\right] \quad \text{for an unsuccessful search}$$

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing

- Quadratic probing and double hashing – average number of comparisons

$$\frac{-\log_e(1-\alpha)}{\alpha}$$ for a successful search, and

$$\frac{1}{1-\alpha}$$ for an unsuccessful search

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing

- Efficiency of the retrieval and removal operations under the separate-chaining approach

$1 + \frac{\alpha}{2}$	for a successful search, and
α	for an unsuccessful search

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing

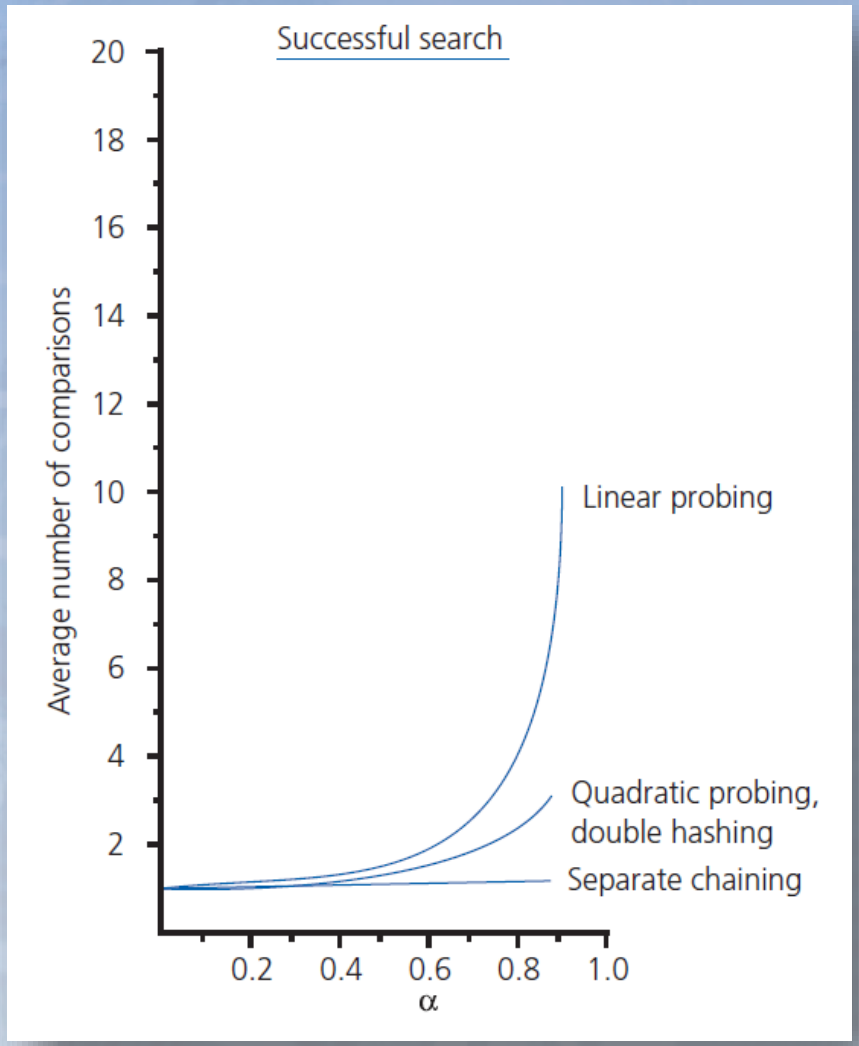


FIGURE 18-15 The relative efficiency of four collision-resolution methods

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing

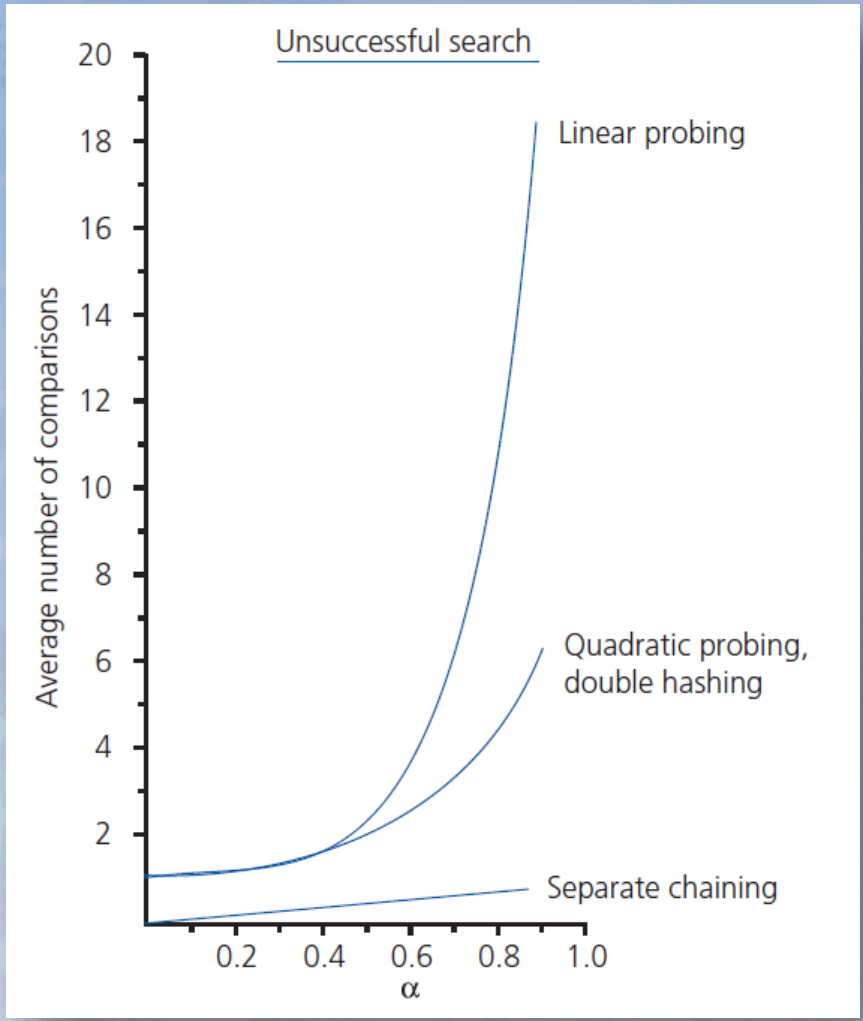


FIGURE 18-15 The relative efficiency of four collision-resolution methods

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Using Hashing, Separate Chaining to Implement ADT Dictionary

FIGURE 18-16 A dictionary entry when separate chaining is used

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Using Hashing, Separate Chaining to Implement ADT Dictionary

```
/** A class of entry objects for a hashing implementation of the
    ADT dictionary.
 @file HashedEntry.h */

#ifndef HASHED_ENTRY_
#define HASHED_ENTRY_

#include "Entry.h"

template<class KeyType, class ValueType>
class HashedEntry : public Entry<KeyType, ValueType>
{
private:
   std::shared_ptr<HashedEntry<KeyType, ValueType>> nextPtr;
public:
```

LISTING 18-5 The class HashedEntry

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Using Hashing, Separate Chaining to Implement ADT Dictionary

```
   std::shared_ptr<HashedEntry<KeyType, ValueType>> nextPtr;
public:
   HashedEntry();
   HashedEntry(KeyType searchKey, ValueType newValue);
   HashedEntry(KeyType searchKey, ValueType newValue,
               std::shared_ptr<HashedEntry<KeyType, ValueType>> nextEntryPtr);

   void setNext(std::shared_ptr<HashedEntry<KeyType, ValueType>>
                                          nextEntryPtr nextEntryPtr);
   auto getNext() const;
}; // end HashedEntry

#include "HashedEntry.cpp"
#endif
```

LISTING 18-5 The class HashedEntry

© 2017 Pearson Education, Hoboken, NJ. All rights reserved